

Live interview pack

State Farm, Bloomington. Second-year intern. Two questions.

Read the shaded paragraph. Stop. The next challenge waits until that one works.

These openers follow the hash-map and string shape described for State Farm Java screens. The three challenges after each opener are ours. Do not say the company asked them. No AI. Plain text editor.

What you listen for. They restate the example before typing. They name the pass that preserves order. They keep a rule in one method so the next change does not copy it. They can say how long a remembered answer takes.

Question 1. First character that appears once

Read this

You get one line of text, the kind of short note someone might type on a claim. Find the first character that appears exactly once. Read left to right, and count every character, including spaces and punctuation. If no character appears exactly once, say so plainly instead of guessing. Before you write code, walk through the line aabbcddeeff and say which character you would return and why. Then say how the time grows as the line gets longer.

Solution

Count every character, then walk the original line again and return the first character whose count is 1. The second walk keeps left-to-right order. On aabbcddeeff the answer is c. Time grows with the length of the line. Memory grows with the number of distinct characters.

```
public static Character firstUnique(String line) {
    if (line == null) throw new IllegalArgumentException("line is required");
    Map<Character, Integer> counts = new HashMap<>();
    for (int i = 0; i < line.length(); i++) {
        counts.merge(line.charAt(i), 1, Integer::sum);
    }
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (counts.get(c) == 1) return c;
    }
    return null;
}
```

Challenge 1. Letters only

Read this

Only letters count, and uppercase and lowercase are the same letter. Skip spaces and punctuation. Return the character as it first appeared, with its original capitalization. The line A a b B c! should give c.

Solution

Fold case only for the count key. Skip non-letters in both walks. Return the character from the line, not the lowercased key.

Challenge 2. One job per method

Read this

Split the work. Counting is its own method. Finding the answer is another. A missing line is an error, not an answer. No unique character is an answer, and it must not look like a missing line.

Solution

A null line throws `IllegalArgumentException`. An empty `Optional` means none, so it cannot be confused with a missing argument. The finder calls the counter. It does not recount.

Challenge 3. Remember a few lines

Read this

I will ask about the same line again. Answer from memory. You may remember only a fixed number of lines, say 100. When you are full, forget the line you have used least. Remembering that a line had no unique character is still an answer. It is not the same as never having seen the line.

Solution

A `LinkedHashMap` built with access order, and `removeEldestEntry` when the size passes the cap, drops the least recently used line. Store `Optional.empty` for no unique character, so a saved answer is not mistaken for a miss. `containsKey` does not count as use. `get` does. A hit does not rescan. A miss scans twice. Memory is bounded by the cap.

Question 2. Words that use the same letters

Read this

You get a list of words. Put words in the same group when they use the same letters the same number of times, only rearranged. tap and pat go together. tap and taps do not. Inside a group, keep the words in the order you first saw them. Keep the groups in the order of the first word that started each group. An empty list means no groups. Walk through tap, pat, taps, apt before you code, and say how the time grows as the list gets longer.

Solution

Give each word a signature from its letter counts, written in sorted letter order with separators so counts cannot blur together. Group by that signature. A linked map keeps groups in first-seen order. The sample is tap, pat, apt and then taps. Time grows with the total number of characters. Sorting is only the distinct letters of each word.

```
public static List<List<String>> groupSameLetters(List<String> words) {
    if (words == null) throw new IllegalArgumentException("words is required");
    Map<String, List<String>> groups = new LinkedHashMap<>();
    for (String word : words) {
        if (word == null) throw new IllegalArgumentException("word is required");
        groups.computeIfAbsent(signature(word), k -> new ArrayList<>()).add(word);
    }
    return new ArrayList<>(groups.values());
}

private static String signature(String word) {
    Map<Character, Integer> counts = new HashMap<>();
    for (int i = 0; i < word.length(); i++) {
        counts.merge(word.charAt(i), 1, Integer::sum);
    }
    List<Character> letters = new ArrayList<>(counts.keySet());
    Collections.sort(letters);
    StringBuilder key = new StringBuilder();
    for (char c : letters) key.append(c).append('#').append(counts.get(c)).append('|');
    return key.toString();
}
```

Challenge 1. Ignore noise

Read this

Ignore capitalization, and ignore any character that is not a letter. Tap! and pat land in the same group.

Solution

Lowercase only letters, and skip everything else, before counting. The signature method is the only place that changes.

Challenge 2. The rule lives in one place

Read this

The rule that decides two words match must live in one method. Grouping only calls it. If tomorrow digits stop counting, that is one edit, not a hunt through the file.

Solution

signature is that method. groupSameLetters does not count characters itself. A later rule change stays inside signature.

Challenge 3. Do not recount a word

Read this

The same word may show up in a later batch. Remember its signature. Keep only a fixed number of words in memory. When you are full, forget the word you have used least.

Solution

Same linked map as question 1, access order, drop the eldest when over the cap. Key the memory by the raw word, so a repeat is not recounted. A hit is instant. A miss still counts the letters once.

Stop after challenge 2 if the solution is still messy. Challenge 3 is for someone who finished early and can talk about time and memory without being asked.